MultiOutput K-Nearest Neighbors (KNN) Model: Yearly Analysis and Predictions

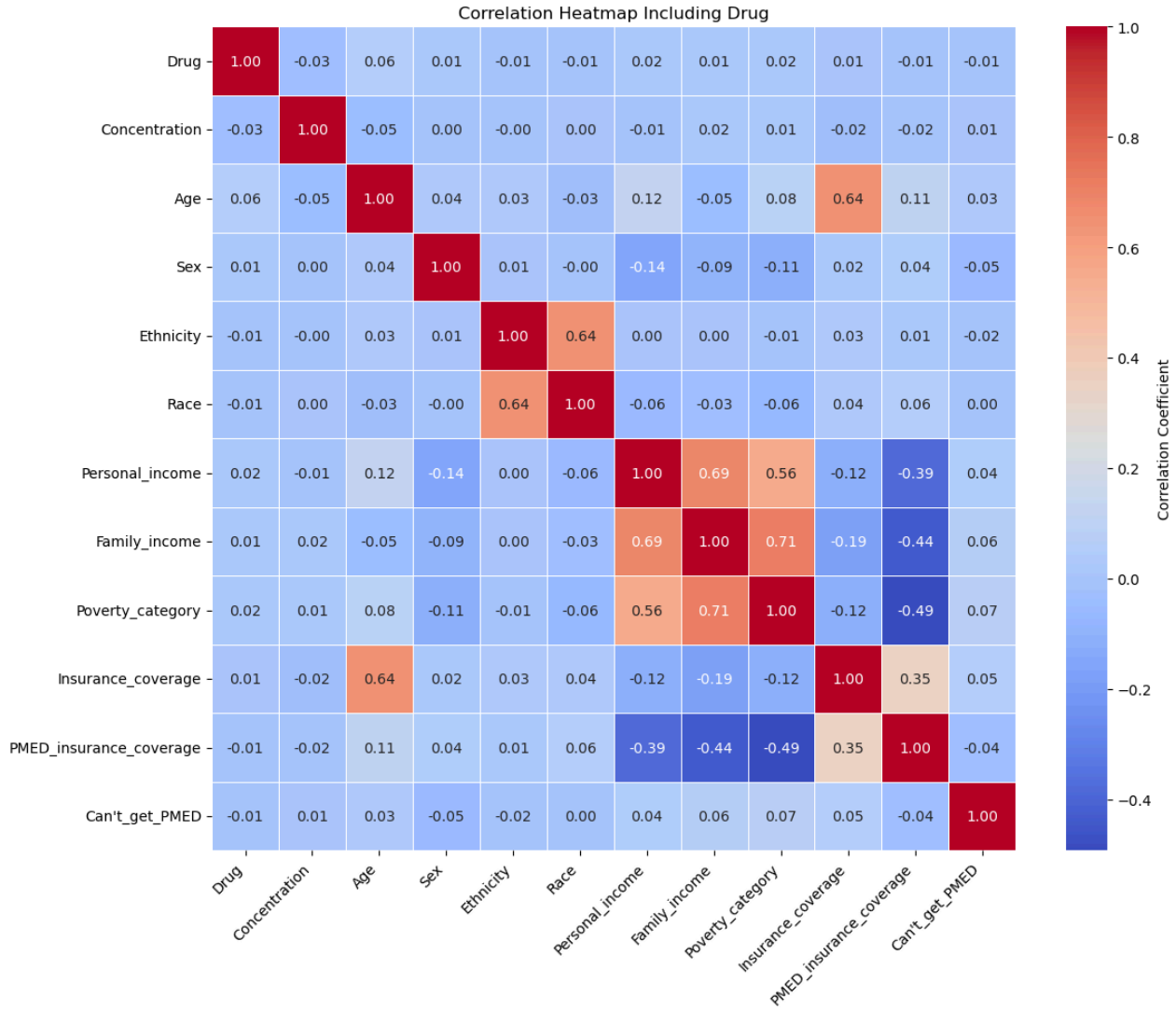
```
import pandas as pd
pd.options.mode.chained_assignment = None
```

MultiOutput KNN model of Pharmasuticals Data of 2014

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, f1_score, confusion_matrix, recall_scor
from sklearn.preprocessing import StandardScaler, LabelEncoder
import numpy as np

# Load dataset
df = pd.read_csv("meps_clean_2014.csv")

# Define features (X) and target variables (y)
X = df[['Age', 'Sex', 'Ethnicity', 'Race', 'Personal_income', 'Family_income',
        'Poverty_category', 'Insurance_coverage', 'PMED_insurance_coverage', 'Can\'t_get_PMED']]
y = df[['Concentration', 'Drug']]

feature_set = ['Drug', 'Concentration', 'Age', 'Sex', 'Ethnicity',
               'Race', 'Personal_income', 'Family_income',
               'Poverty_category', 'Insurance_coverage',
               'PMED_insurance_coverage', 'Can\'t_get_PMED']

# Drop rows with missing values in any of the features or target variables
df_cleaned = df.dropna(subset=feature_set)

# Handle categorical columns in X (features) and convert them to numeric using LabelEncoder
categorical_columns = X.select_dtypes(include=['object']).columns
for col in categorical_columns:
    X[col] = LabelEncoder().fit_transform(X[col])

# Handle missing values in target variables y (Concentration and Drug)
encoder = LabelEncoder()
y['Drug'] = encoder.fit_transform(y['Drug'])

# Replace infinite values with NaN
y.replace([np.inf, -np.inf], np.nan, inplace=True)

# Convert Concentration to numeric values and handle any NaN values
y['Concentration'] = pd.to_numeric(y['Concentration'], errors='coerce')

# Ensure there are no missing values in features or targets after conversion
X = X.fillna(0)
y = y.fillna(0)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize the KNN Regressor
knn_model = KNeighborsRegressor(n_neighbors=5)
multi_output_knn = MultiOutputRegressor(knn_model)

# Train the model
multi_output_knn.fit(X_train_scaled, y_train)

# Make predictions
y_pred = multi_output_knn.predict(X_test_scaled)

# Evaluate Concentration (regression)
mse = mean_squared_error(y_test['Concentration'], y_pred[:, 0])
r2 = r2_score(y_test['Concentration'], y_pred[:, 0])

# Print metrics for Concentration
print("\nRegression Metrics for Concentration:")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R² Score: {r2}")
```

```
# Evaluate Drug (classification)
threshold = 0.5
y_test_binarized = (y_test['Drug'] >= threshold).astype(int)
y_pred_binarized = (y_pred[:, 1] >= threshold).astype(int)

conf_matrix = confusion_matrix(y_test_binarized, y_pred_binarized)
accuracy = accuracy_score(y_test_binarized, y_pred_binarized)
f1 = f1_score(y_test_binarized, y_pred_binarized)
recall = recall_score(y_test_binarized, y_pred_binarized)

# Print metrics for Drug
print("\nClassification Metrics for Drug:")
print(f"Accuracy: {accuracy}")
print(f"F1 Score: {f1}")
print(f"Recall: {recall}")
print("\nConfusion Matrix:")
print(conf_matrix)

# Create a DataFrame to compare actual vs predicted values
comparison_df = pd.DataFrame({
    'Actual_Concentration': y_test['Concentration'],
    'Predicted_Concentration': y_pred[:, 0],
    'Actual_Drug': y_test['Drug'],
    'Predicted_Drug': y_pred[:, 1]
})

# Add inverse-transformed Drug names
comparison_df['Predicted_Drug_Name'] = encoder.inverse_transform(comparison_df['Predicted_Drug'].round().astype(i
comparison_df['Actual_Drug_Name'] = encoder.inverse_transform(comparison_df['Actual_Drug'].round().astype(int))

# Save results to CSV
comparison_df.to_csv("comparison_results_2014.csv", index=False)

# Display comparison table
print("\nComparison of Actual vs Predicted Values:")
print(comparison_df.head())

# Ensure 'Drug' is numeric and included in the cleaned dataset
df_cleaned['Drug'] = encoder.transform(df_cleaned['Drug'])

# Correlation matrix and heatmap (including Drug)
correlation_matrix = df_cleaned[feature_set].corr()

# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5,
            cbar_kws={'label': 'Correlation Coefficient'})
plt.title("Correlation Heatmap Including Drug")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
Regression Metrics for Concentration:
Mean Squared Error (MSE): 1.0988431161012617
R² Score: 0.13494099933483272

Classification Metrics for Drug:
Accuracy: 0.9932475292095517
F1 Score: 0.996612118349999
Recall: 0.9999175971323802

Confusion Matrix:
[[    3   326]
 [    4 48538]]

Comparison of Actual vs Predicted Values:
        Actual_Concentration  Predicted_Concentration  Actual_Drug  \
217240              0.000024                 0.000853          145
216189              0.000739                 0.000582          274
179156              0.000068                 0.000068          130
192999              0.001450                 0.001693          334
220253              0.000008                 0.000045          113

        Predicted_Drug Predicted_Drug_Name Actual_Drug_Name
217240           162.2      INSULIN LISPRO       GUANFACINE
216189           270.6         PROPAFENONE       QUETIAPINE
179156           188.6          LOPERAMIDE     FLUOCINONIDE
192999           194.0          LOVASTATIN        VERAPAMIL
220253           146.2         HALOPERIDOL        ESTRADIOL
```

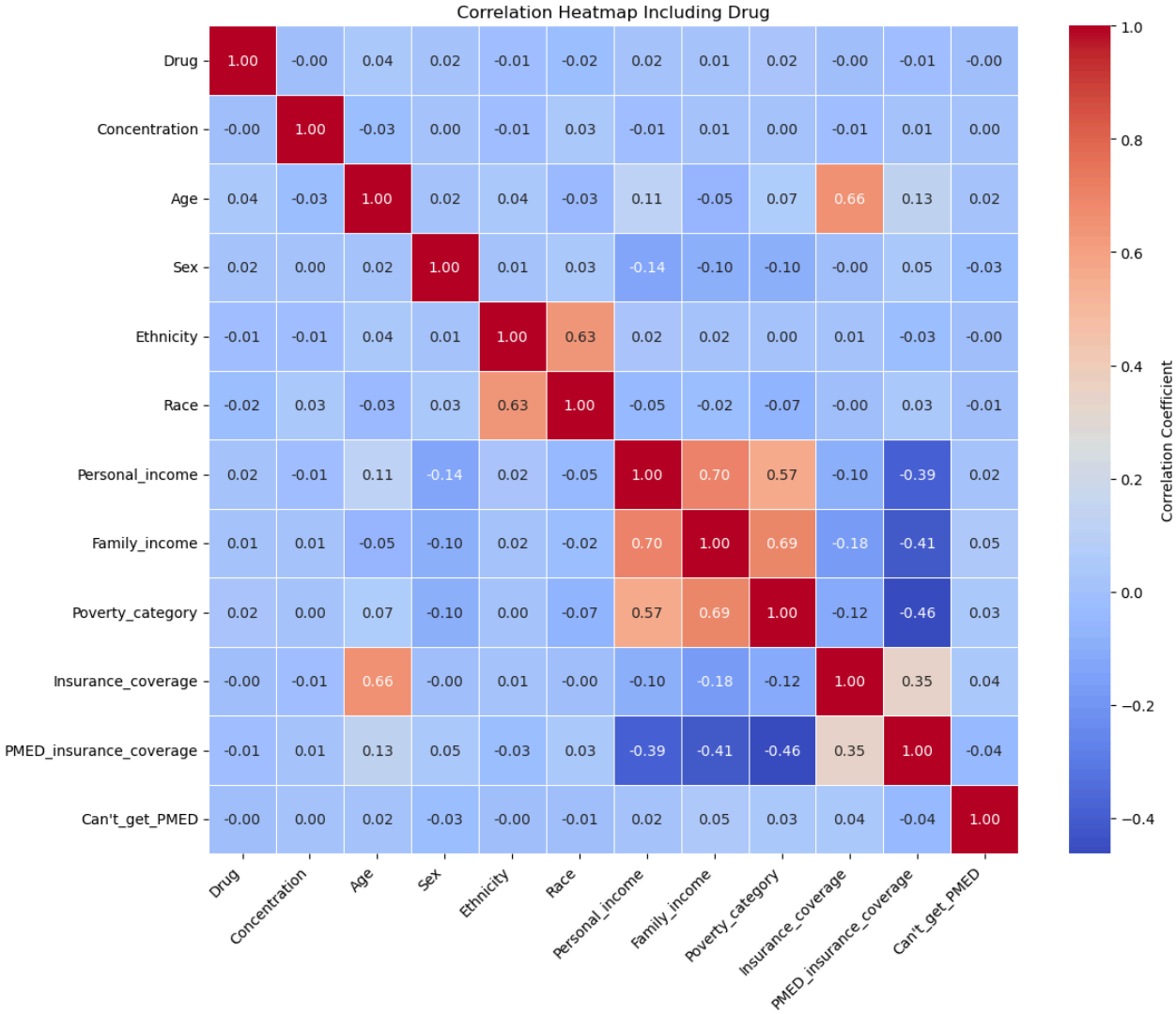


MultiOutput KNN model of Pharmasuticals Data of 2016

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, f1_score, confusion_matrix, recall_sco
from sklearn.preprocessing import StandardScaler, LabelEncoder
import numpy as np

# Load dataset
df = pd.read_csv("meps_clean_2016.csv")

# Define features (X) and target variables (y)
X = df[['Age', 'Sex', 'Ethnicity', 'Race', 'Personal_income', 'Family_income',
        'Poverty_category', 'Insurance_coverage', 'PMED_insurance_coverage', 'Can\'t_get_PMED']]
y = df[['Concentration', 'Drug']]

feature_set = ['Drug', 'Concentration', 'Age', 'Sex', 'Ethnicity',
               'Race', 'Personal_income', 'Family_income',
               'Poverty_category', 'Insurance_coverage',
               'PMED_insurance_coverage', 'Can\'t_get_PMED']

# Drop rows with missing values in any of the features or target variables
df_cleaned = df.dropna(subset=feature_set)

# Handle categorical columns in X (features) and convert them to numeric using LabelEncoder
categorical_columns = X.select_dtypes(include=['object']).columns
for col in categorical_columns:
    X[col] = LabelEncoder().fit_transform(X[col])

# Handle missing values in target variables y (Concentration and Drug)
encoder = LabelEncoder()
y['Drug'] = encoder.fit_transform(y['Drug'])

# Replace infinite values with NaN
y.replace([np.inf, -np.inf], np.nan, inplace=True)

# Convert Concentration to numeric values and handle any NaN values
y['Concentration'] = pd.to_numeric(y['Concentration'], errors='coerce')

# Ensure there are no missing values in features or targets after conversion
X = X.fillna(0)
y = y.fillna(0)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize the KNN Regressor
knn_model = KNeighborsRegressor(n_neighbors=5)
multi_output_knn = MultiOutputRegressor(knn_model)

# Train the model
multi_output_knn.fit(X_train_scaled, y_train)

# Make predictions
y_pred = multi_output_knn.predict(X_test_scaled)

# Evaluate Concentration (regression)
mse = mean_squared_error(y_test['Concentration'], y_pred[:, 0])
r2 = r2_score(y_test['Concentration'], y_pred[:, 0])

# Print metrics for Concentration
print("\nRegression Metrics for Concentration:")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R² Score: {r2}")

# Evaluate Drug (classification)
threshold = 0.5
y_test_binarized = (y_test['Drug'] >= threshold).astype(int)
y_pred_binarized = (y_pred[:, 1] >= threshold).astype(int)

conf_matrix = confusion_matrix(y_test_binarized, y_pred_binarized)
accuracy = accuracy_score(y_test_binarized, y_pred_binarized)
f1 = f1_score(y_test_binarized, y_pred_binarized)
recall = recall_score(y_test_binarized, y_pred_binarized)
```

```
# Print metrics for Drug
print("\nClassification Metrics for Drug:")
print(f"Accuracy: {accuracy}")
print(f"F1 Score: {f1}")
print(f"Recall: {recall}")
print("\nConfusion Matrix:")
print(conf_matrix)

# Create a DataFrame to compare actual vs predicted values
comparison_df = pd.DataFrame({
    'Actual_Concentration': y_test['Concentration'],
    'Predicted_Concentration': y_pred[:, 0],
    'Actual_Drug': y_test['Drug'],
    'Predicted_Drug': y_pred[:, 1]
})

# Add inverse-transformed Drug names
comparison_df['Predicted_Drug_Name'] = encoder.inverse_transform(comparison_df['Predicted_Drug'].round().astype(
comparison_df['Actual_Drug_Name'] = encoder.inverse_transform(comparison_df['Actual_Drug'].round().astype(int))

# Save results to CSV
comparison_df.to_csv("comparison_results_2016.csv", index=False)

# Display comparison table
print("\nComparison of Actual vs Predicted Values:")
print(comparison_df.head())

# Ensure 'Drug' is numeric and included in the cleaned dataset
df_cleaned['Drug'] = encoder.transform(df_cleaned['Drug'])

# Correlation matrix and heatmap (including Drug)
correlation_matrix = df_cleaned[feature_set].corr()

# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5,
            cbar_kws={'label': 'Correlation Coefficient'})
plt.title("Correlation Heatmap Including Drug")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_484/1453418351.py:12: DtypeWarning:

Columns (22) have mixed types. Specify dtype option on import or set low_memory=False.


Regression Metrics for Concentration:
Mean Squared Error (MSE): 0.031499061769693186
R² Score: 0.14356985277355994

Classification Metrics for Drug:
Accuracy: 0.9938071780436313
F1 Score: 0.9968939714810109
Recall: 0.9999797685522376

Confusion Matrix:
[[    0   307]
 [    1 49427]]

Comparison of Actual vs Predicted Values:
        Actual_Concentration  Predicted_Concentration  Actual_Drug  \
88852           5.219005e-07                 0.000113            4
170870          8.054020e-05                 0.000202          185
224711          2.013505e-05                 0.000113          171
66285           2.965910e-04                 0.009779          235
223412          2.416206e-04                 0.000339          248

        Predicted_Drug Predicted_Drug_Name Actual_Drug_Name
88852             84.4   DEXTROAMPHETAMINE        ALBUTEROL
170870           180.4         LINACLOTIDE       LORATADINE
224711           116.4          FENOPROFEN        LETROZOLE
66285            206.2          METOPROLOL       OMEPRAZOLE
223412           158.4          IRBESARTAN      PHENTERMINE
```

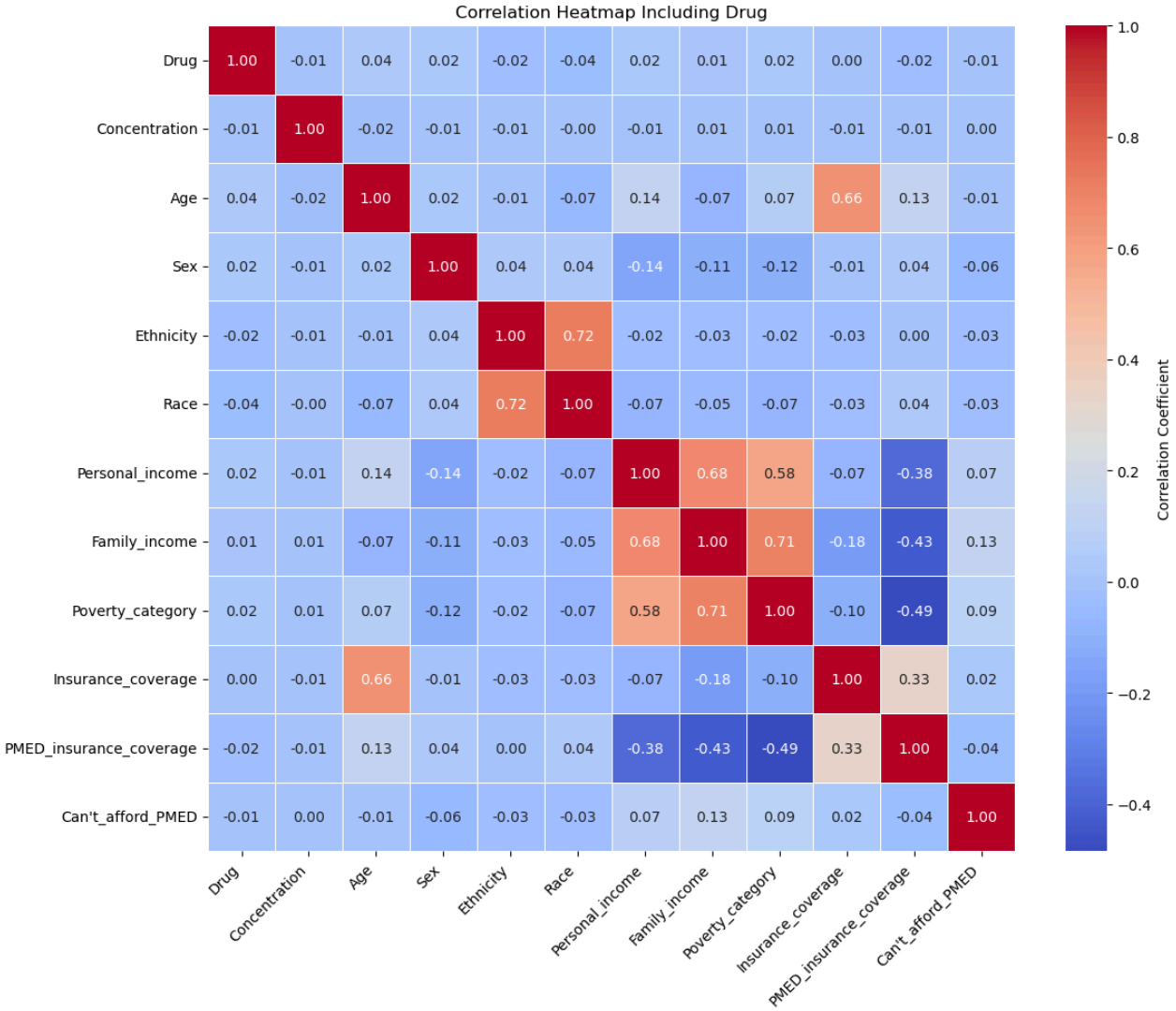


MultiOutput KNN-PCA model of Pharmasuticals Data of 2018

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, f1_score, confusion_matrix, recall_sco
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
import numpy as np

# Load dataset
df = pd.read_csv("meps_clean_2018.csv")

# Define features (X) and target variables (y)
X = df[['Age', 'Sex', 'Ethnicity', 'Race', 'Personal_income', 'Family_income',
        'Poverty_category', 'Insurance_coverage', 'PMED_insurance_coverage', 'Can\'t_afford_PMED']]
y = df[['Concentration', 'Drug']]

feature_set = ['Drug', 'Concentration', 'Age', 'Sex', 'Ethnicity',
               'Race', 'Personal_income', 'Family_income',
               'Poverty_category', 'Insurance_coverage',
               'PMED_insurance_coverage', 'Can\'t_afford_PMED']

# Drop rows with missing values in any of the features or target variables
df_cleaned = df.dropna(subset=feature_set)

# Handle categorical columns in X (features) and convert them to numeric using LabelEncoder
categorical_columns = X.select_dtypes(include=['object']).columns
for col in categorical_columns:
    X[col] = LabelEncoder().fit_transform(X[col])

# Handle missing values in target variables y (Concentration and Drug)
encoder = LabelEncoder()
y['Drug'] = encoder.fit_transform(y['Drug'])

# Replace infinite values with NaN
y.replace([np.inf, -np.inf], np.nan, inplace=True)

# Convert Concentration to numeric values and handle any NaN values
y['Concentration'] = pd.to_numeric(y['Concentration'], errors='coerce')

# Ensure there are no missing values in features or targets after conversion
X = X.fillna(0)
y = y.fillna(0)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Apply PCA for dimensionality reduction
pca = PCA(n_components=2)  # Reduce to 2 components for visualization
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Initialize the KNN Regressor
knn_model = KNeighborsRegressor(n_neighbors=8)
multi_output_knn = MultiOutputRegressor(knn_model)

# Train the model
multi_output_knn.fit(X_train_pca, y_train)

# Make predictions
y_pred = multi_output_knn.predict(X_test_pca)

# Evaluate Concentration (regression)
mse = mean_squared_error(y_test['Concentration'], y_pred[:, 0])
r2 = r2_score(y_test['Concentration'], y_pred[:, 0])

# Print metrics for Concentration
print("\nRegression Metrics for Concentration:")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R² Score: {r2}")

# Evaluate Drug (classification)
```

```
threshold = 0.5
y_test_binarized = (y_test['Drug'] >= threshold).astype(int)
y_pred_binarized = (y_pred[:, 1] >= threshold).astype(int)

conf_matrix = confusion_matrix(y_test_binarized, y_pred_binarized)
accuracy = accuracy_score(y_test_binarized, y_pred_binarized)
f1 = f1_score(y_test_binarized, y_pred_binarized)
recall = recall_score(y_test_binarized, y_pred_binarized)

# Print metrics for Drug
print("\nClassification Metrics for Drug:")
print(f"Accuracy: {accuracy}")
print(f"F1 Score: {f1}")
print(f"Recall: {recall}")
print("\nConfusion Matrix:")
print(conf_matrix)

# Create a DataFrame to compare actual vs predicted values
comparison_df = pd.DataFrame({
    'Actual_Concentration': y_test['Concentration'],
    'Predicted_Concentration': y_pred[:, 0],
    'Actual_Drug': y_test['Drug'],
    'Predicted_Drug': y_pred[:, 1]
})

# Add inverse-transformed Drug names
comparison_df['Predicted_Drug_Name'] = encoder.inverse_transform(comparison_df['Predicted_Drug'].round().astype(
comparison_df['Actual_Drug_Name'] = encoder.inverse_transform(comparison_df['Actual_Drug'].round().astype(int))

# Save results to CSV
comparison_df.to_csv("comparison_results_2018.csv", index=False)

# Display comparison table
print("\nComparison of Actual vs Predicted Values:")
print(comparison_df.head())

# Ensure 'Drug' is numeric and included in the cleaned dataset
df_cleaned['Drug'] = encoder.transform(df_cleaned['Drug'])

# Correlation matrix and heatmap (including Drug)
correlation_matrix = df_cleaned[feature_set].corr()

# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5,
            cbar_kws={'label': 'Correlation Coefficient'})
plt.title("Correlation Heatmap Including Drug")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_484/929319801.py:13: DtypeWarning:

Columns (22) have mixed types. Specify dtype option on import or set low_memory=False.


Regression Metrics for Concentration:
Mean Squared Error (MSE): 0.433435517882807
R² Score: 0.19569919608810304

Classification Metrics for Drug:
Accuracy: 0.9935820423648435
F1 Score: 0.9967801798658144
Recall: 0.9999681761766859

Confusion Matrix:
[[    5   202]
 [    1 31422]]

Comparison of Actual vs Predicted Values:
        Actual_Concentration  Predicted_Concentration  Actual_Drug  \
131415              0.000016                 0.002990          227
125377              0.005234                 0.002693          135
54331               0.000322                 0.002512          184
110912              0.000161                 0.000326          134
44975               0.000805                 0.001100          188

        Predicted_Drug Predicted_Drug_Name Actual_Drug_Name
131415         149.500         HYDROXYZINE    NITROGLYCERIN
125377         155.250           IMIQUIMOD       GABAPENTIN
54331          136.625          GENTAMICIN       LISINOPRIL
110912         170.125         LATANOPROST       FUROSEMIDE
44975          115.750           ESTRADIOL         LOSARTAN
```

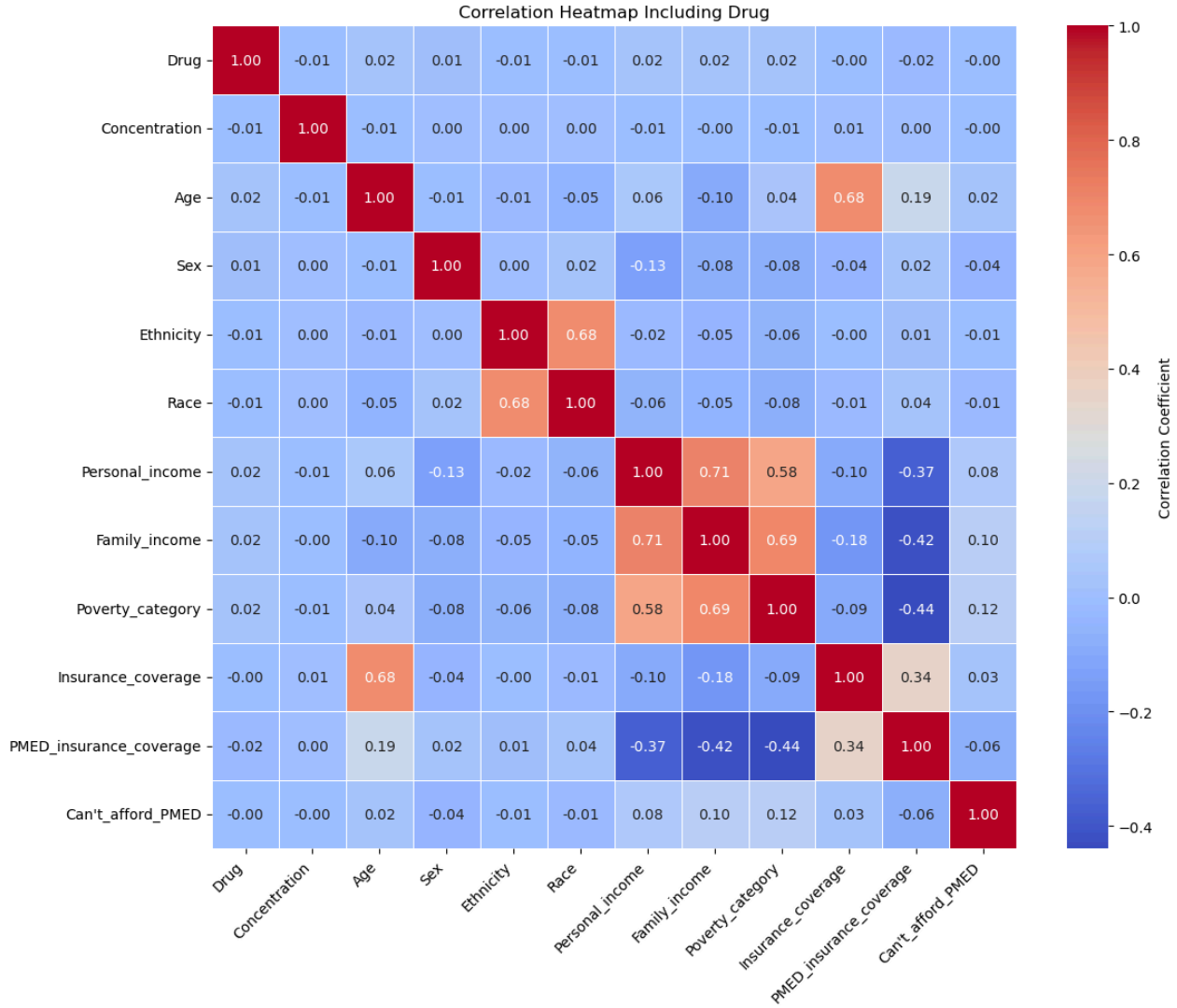


MultiOutput KNN-PCA Model for Pharmaceutical Data of 2020

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, f1_score, confusion_matrix, recall_sco
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
import numpy as np

# Load dataset
df = pd.read_csv("meps_clean_2020.csv")

# Define features (X) and target variables (y)
X = df[['Age', 'Sex', 'Ethnicity', 'Race', 'Personal_income', 'Family_income',
        'Poverty_category', 'Insurance_coverage', 'PMED_insurance_coverage', 'Can\'t_afford_PMED']]
y = df[['Concentration', 'Drug']]

feature_set = ['Drug', 'Concentration', 'Age', 'Sex', 'Ethnicity',
               'Race', 'Personal_income', 'Family_income',
               'Poverty_category', 'Insurance_coverage',
               'PMED_insurance_coverage', 'Can\'t_afford_PMED']

# Drop rows with missing values in any of the features or target variables
df_cleaned = df.dropna(subset=feature_set)

# Handle categorical columns in X (features) and convert them to numeric using LabelEncoder
categorical_columns = X.select_dtypes(include=['object']).columns
for col in categorical_columns:
    X[col] = LabelEncoder().fit_transform(X[col])

# Handle missing values in target variables y (Concentration and Drug)
encoder = LabelEncoder()
y['Drug'] = encoder.fit_transform(y['Drug'])

# Replace infinite values with NaN
y.replace([np.inf, -np.inf], np.nan, inplace=True)

# Convert Concentration to numeric values and handle any NaN values
y['Concentration'] = pd.to_numeric(y['Concentration'], errors='coerce')

# Ensure there are no missing values in features or targets after conversion
X = X.fillna(0)
y = y.fillna(0)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Apply PCA for dimensionality reduction
pca = PCA(n_components=2)  # Reduce to 2 components for visualization
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Initialize the KNN Regressor
knn_model = KNeighborsRegressor(n_neighbors=8)
multi_output_knn = MultiOutputRegressor(knn_model)

# Train the model
multi_output_knn.fit(X_train_pca, y_train)

# Make predictions
y_pred = multi_output_knn.predict(X_test_pca)

# Evaluate Concentration (regression)
mse = mean_squared_error(y_test['Concentration'], y_pred[:, 0])
r2 = r2_score(y_test['Concentration'], y_pred[:, 0])

# Print metrics for Concentration
print("\nRegression Metrics for Concentration:")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R² Score: {r2}")

# Evaluate Drug (classification)
```

```
threshold = 0.5
y_test_binarized = (y_test['Drug'] >= threshold).astype(int)
y_pred_binarized = (y_pred[:, 1] >= threshold).astype(int)

conf_matrix = confusion_matrix(y_test_binarized, y_pred_binarized)
accuracy = accuracy_score(y_test_binarized, y_pred_binarized)
f1 = f1_score(y_test_binarized, y_pred_binarized)
recall = recall_score(y_test_binarized, y_pred_binarized)

# Print metrics for Drug
print("\nClassification Metrics for Drug:")
print(f"Accuracy: {accuracy}")
print(f"F1 Score: {f1}")
print(f"Recall: {recall}")
print("\nConfusion Matrix:")
print(conf_matrix)

# Create a DataFrame to compare actual vs predicted values
comparison_df = pd.DataFrame({
    'Actual_Concentration': y_test['Concentration'],
    'Predicted_Concentration': y_pred[:, 0],
    'Actual_Drug': y_test['Drug'],
    'Predicted_Drug': y_pred[:, 1]
})

# Add inverse-transformed Drug names
comparison_df['Predicted_Drug_Name'] = encoder.inverse_transform(comparison_df['Predicted_Drug'].round().astype(
comparison_df['Actual_Drug_Name'] = encoder.inverse_transform(comparison_df['Actual_Drug'].round().astype(int))

# Save results to CSV
comparison_df.to_csv("comparison_results_2020.csv", index=False)

# Display comparison table
print("\nComparison of Actual vs Predicted Values:")
print(comparison_df.head())

# Ensure 'Drug' is numeric and included in the cleaned dataset
df_cleaned['Drug'] = encoder.transform(df_cleaned['Drug'])

# Correlation matrix and heatmap (including Drug)
correlation_matrix = df_cleaned[feature_set].corr()

# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5,
            cbar_kws={'label': 'Correlation Coefficient'})
plt.title("Correlation Heatmap Including Drug")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_484/2239775387.py:13: DtypeWarning:

Columns (20) have mixed types. Specify dtype option on import or set low_memory=False.


Regression Metrics for Concentration:
Mean Squared Error (MSE): 0.005325730519073026
R² Score: 0.024719509422248542

Classification Metrics for Drug:
Accuracy: 0.9964254617111956
F1 Score: 0.998209530805252
Recall: 1.0

Confusion Matrix:
[[    0   156]
 [    0 43486]]

Comparison of Actual vs Predicted Values:
        Actual_Concentration  Predicted_Concentration  Actual_Drug  \
44725               0.000040                 0.000086          173
187359              0.000003                 0.000333          213
179514              0.000488                 0.003585           19
179371              0.000201                 0.000081           19
112089              0.000322                 0.000215          139

        Predicted_Drug Predicted_Drug_Name Actual_Drug_Name
44725          190.625     METHYLPHENIDATE       LISINOPRIL
187359         192.750      METOCLOPRAMIDE    NITROGLYCERIN
179514         192.125  METHYLPREDNISOLONE         ATENOLOL
179371         110.625         FENOFIBRATE         ATENOLOL
112089          89.625           DONEPEZIL      HYDROXYZINE
```

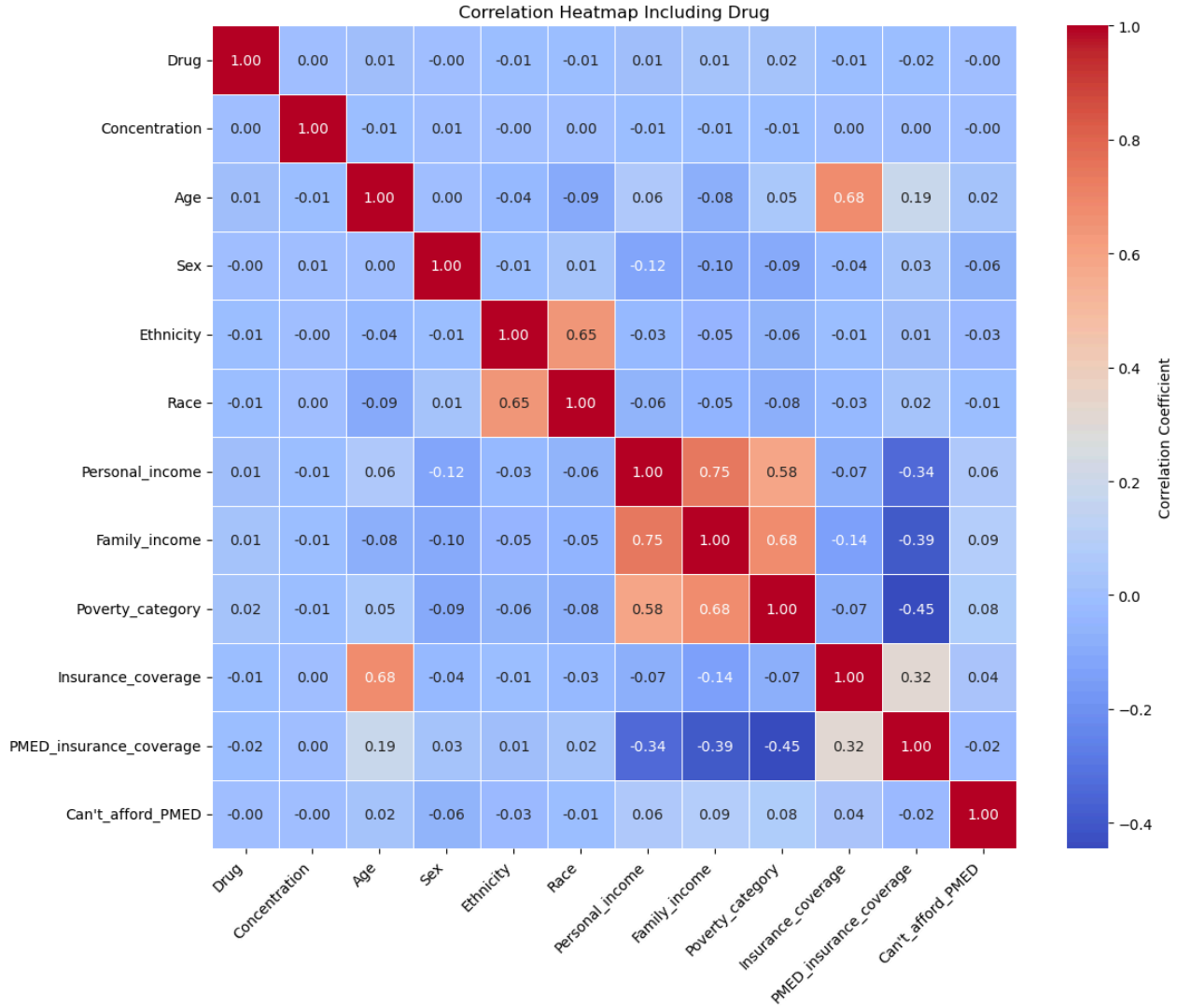


MultiOutput KNN-PCA Model for Pharmaceutical Data of 2021

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, f1_score, confusion_matrix, recall_sco
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
import numpy as np

# Load dataset
df = pd.read_csv("meps_clean_2021.csv")

# Define features (X) and target variables (y)
X = df[['Age', 'Sex', 'Ethnicity', 'Race', 'Personal_income', 'Family_income',
        'Poverty_category', 'Insurance_coverage', 'PMED_insurance_coverage', 'Can\'t_afford_PMED']]
y = df[['Concentration', 'Drug']]

feature_set = ['Drug', 'Concentration', 'Age', 'Sex', 'Ethnicity',
               'Race', 'Personal_income', 'Family_income',
               'Poverty_category', 'Insurance_coverage',
               'PMED_insurance_coverage', 'Can\'t_afford_PMED']

# Drop rows with missing values in any of the features or target variables
df_cleaned = df.dropna(subset=feature_set)

# Handle categorical columns in X (features) and convert them to numeric using LabelEncoder
categorical_columns = X.select_dtypes(include=['object']).columns
for col in categorical_columns:
    X[col] = LabelEncoder().fit_transform(X[col])

# Handle missing values in target variables y (Concentration and Drug)
encoder = LabelEncoder()
y['Drug'] = encoder.fit_transform(y['Drug'])

# Replace infinite values with NaN
y.replace([np.inf, -np.inf], np.nan, inplace=True)

# Convert Concentration to numeric values and handle any NaN values
y['Concentration'] = pd.to_numeric(y['Concentration'], errors='coerce')

# Ensure there are no missing values in features or targets after conversion
X = X.fillna(0)
y = y.fillna(0)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Apply PCA for dimensionality reduction
pca = PCA(n_components=2)  # Reduce to 2 components for visualization
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Initialize the KNN Regressor
knn_model = KNeighborsRegressor(n_neighbors=9)
multi_output_knn = MultiOutputRegressor(knn_model)

# Train the model
multi_output_knn.fit(X_train_pca, y_train)

# Make predictions
y_pred = multi_output_knn.predict(X_test_pca)

# Evaluate Concentration (regression)
mse = mean_squared_error(y_test['Concentration'], y_pred[:, 0])
r2 = r2_score(y_test['Concentration'], y_pred[:, 0])

# Print metrics for Concentration
print("\nRegression Metrics for Concentration:")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R² Score: {r2}")

# Evaluate Drug (classification)
```

```
threshold = 0.5
y_test_binarized = (y_test['Drug'] >= threshold).astype(int)
y_pred_binarized = (y_pred[:, 1] >= threshold).astype(int)

conf_matrix = confusion_matrix(y_test_binarized, y_pred_binarized)
accuracy = accuracy_score(y_test_binarized, y_pred_binarized)
f1 = f1_score(y_test_binarized, y_pred_binarized)
recall = recall_score(y_test_binarized, y_pred_binarized)

# Print metrics for Drug
print("\nClassification Metrics for Drug:")
print(f"Accuracy: {accuracy}")
print(f"F1 Score: {f1}")
print(f"Recall: {recall}")
print("\nConfusion Matrix:")
print(conf_matrix)

# Create a DataFrame to compare actual vs predicted values
comparison_df = pd.DataFrame({
    'Actual_Concentration': y_test['Concentration'],
    'Predicted_Concentration': y_pred[:, 0],
    'Actual_Drug': y_test['Drug'],
    'Predicted_Drug': y_pred[:, 1]
})

# Add inverse-transformed Drug names
comparison_df['Predicted_Drug_Name'] = encoder.inverse_transform(comparison_df['Predicted_Drug'].round().astype(
comparison_df['Actual_Drug_Name'] = encoder.inverse_transform(comparison_df['Actual_Drug'].round().astype(int))

# Save results to CSV
comparison_df.to_csv("comparison_results_2021.csv", index=False)

# Display comparison table
print("\nComparison of Actual vs Predicted Values:")
print(comparison_df.head())

# Ensure 'Drug' is numeric and included in the cleaned dataset
df_cleaned['Drug'] = encoder.transform(df_cleaned['Drug'])

# Correlation matrix and heatmap (including Drug)
correlation_matrix = df_cleaned[feature_set].corr()

# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5,
            cbar_kws={'label': 'Correlation Coefficient'})
plt.title("Correlation Heatmap Including Drug")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_484/1319645556.py:13: DtypeWarning:

Columns (20) have mixed types. Specify dtype option on import or set low_memory=False.


Regression Metrics for Concentration:
Mean Squared Error (MSE): 0.029041810786937145
R² Score: 0.06385060886391658

Classification Metrics for Drug:
Accuracy: 0.9958830614255995
F1 Score: 0.9979372846766584
Recall: 1.0

Confusion Matrix:
[[    0   188]
 [    0 45477]]

Comparison of Actual vs Predicted Values:
        Actual_Concentration  Predicted_Concentration  Actual_Drug  \
58514               0.000040                 0.000121           19
134578              0.000805                 0.003212          151
87135               0.000004                 0.000298          242
142646              0.000081                 0.000271           82
228053              0.000322                 0.003000          128

        Predicted_Drug Predicted_Drug_Name Actual_Drug_Name
58514        97.666667        FLURBIPROFEN     ATORVASTATIN
134578      157.000000           MEMANTINE         LOSARTAN
87135       148.444444          LOPERAMIDE       TAMSULOSIN
142646      111.777778         HYDROCODONE     ESCITALOPRAM
228053      172.888889        MOXIFLOXACIN     ISOTRETINOIN
```



MultiOutput KNN-PCA Model for Pharmaceutical Data of 2022

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, f1_score, confusion_matrix, recall_sco
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
import numpy as np

# Load dataset
df = pd.read_csv("meps_clean_2022.csv")

# Define features (X) and target variables (y)
X = df[['Age', 'Sex', 'Ethnicity', 'Race', 'Personal_income', 'Family_income',
        'Poverty_category', 'Insurance_coverage', 'PMED_insurance_coverage', 'Can\'t_afford_PMED']]
y = df[['Concentration', 'Drug']]

feature_set = ['Drug', 'Concentration', 'Age', 'Sex', 'Ethnicity',
               'Race', 'Personal_income', 'Family_income',
               'Poverty_category', 'Insurance_coverage',
               'PMED_insurance_coverage', 'Can\'t_afford_PMED']

# Drop rows with missing values in any of the features or target variables
df_cleaned = df.dropna(subset=feature_set)

# Handle categorical columns in X (features) and convert them to numeric using LabelEncoder
categorical_columns = X.select_dtypes(include=['object']).columns
for col in categorical_columns:
    X[col] = LabelEncoder().fit_transform(X[col])

# Handle missing values in target variables y (Concentration and Drug)
encoder = LabelEncoder()
y['Drug'] = encoder.fit_transform(y['Drug'])

# Replace infinite values with NaN
y.replace([np.inf, -np.inf], np.nan, inplace=True)

# Convert Concentration to numeric values and handle any NaN values
y['Concentration'] = pd.to_numeric(y['Concentration'], errors='coerce')

# Ensure there are no missing values in features or targets after conversion
X = X.fillna(0)
y = y.fillna(0)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Apply PCA for dimensionality reduction
pca = PCA(n_components=2)  # Reduce to 2 components for visualization
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Initialize the KNN Regressor
knn_model = KNeighborsRegressor(n_neighbors=9)
multi_output_knn = MultiOutputRegressor(knn_model)

# Train the model
multi_output_knn.fit(X_train_pca, y_train)

# Make predictions
y_pred = multi_output_knn.predict(X_test_pca)

# Evaluate Concentration (regression)
mse = mean_squared_error(y_test['Concentration'], y_pred[:, 0])
r2 = r2_score(y_test['Concentration'], y_pred[:, 0])

# Print metrics for Concentration
print("\nRegression Metrics for Concentration:")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R² Score: {r2}")

# Evaluate Drug (classification)
```